APACHE KAFKA TRAINING PROGRAM

Security Module

HANDS-ON LAB GUIDE

ACLs & Authorization

6 Labs | Scenario-Based Exercises | Challenge Assessment

Prerequisites: Kafka with SASL/PLAIN enabled | ACLs activated in server.properties

Author: Rajesh Pasham

Apache Kafka Training Program | 2026

Lab Overview & Environment Setup

Lab #	Title	Duration	Level
Lab 1	Enable ACLs and Verify the Authorizer	20 min	Beginner
Lab 2	Producer ACL — Grant and Verify Write Access	25 min	Beginner
Lab 3	Consumer ACL — Grant Topic + Group Access	30 min	Intermediate
Lab 4	Prefixed ACLs for Multi-Tenant Topics	30 min	Intermediate
Lab 5	Troubleshoot Authorization Failures	35 min	Intermediate
Lab 6	ACL Challenge: Secure a Full Application Stack	40 min	Advanced

Environment Setup

Complete the following setup before starting Lab 1. All labs build on each other.

1. Enable SASL/PLAIN authentication in server.properties:

```
listeners=SASL_PLAINTEXT://localhost:9092
security.inter.broker.protocol=SASL_PLAINTEXT
sasl.mechanism.inter.broker.protocol=PLAIN
sasl.enabled.mechanisms=PLAIN
```

2. Create a JAAS config file at /opt/kafka/config/kafka_server_jaas.conf:

```
KafkaServer {
  org.apache.kafka.common.security.plain.PlainLoginModule required
  username="admin"
  password="admin-secret"
  user_admin="admin-secret"
  user_alice="alice-secret"
  user_bob="bob-secret"
  user_data-pipeline="pipeline-secret"
  user_order-service="order-secret";
};
```

3. Set KAFKA_OPTS to point to the JAAS file:

```
export KAFKA_OPTS="-
Djava.security.auth.login.config=/opt/kafka/config/kafka_server_jaas.conf"
```

4. Enable ACL authorizer in server.properties:

```
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
allow.everyone.if.no.acl.defined=false
super.users=User:admin
```

5. Restart the broker and verify startup

```
kafka-server-start.sh /opt/kafka/config/server.properties
# In another terminal, test admin access:
```

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config /opt/kafka/config/admin-client.properties --list
```

Create admin-client.properties with:

```
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="admin" password="admin-secret";
```

6. Create test topics (as admin):

```
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --create --topic orders --partitions 3 --replication-factor 1
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --create --topic inventory --partitions 3 --replication-factor 1
```

Warning: If you do not have SASL set up, you can still do all labs using --bootstrap-server localhost:9092 without a --command-config file. ACLs will still be created but the access verification steps will show all users as ANONYMOUS.

LAB 1: Enable ACLs and Verify the Authorizer

| Duration: 20 minutes | Level: **Beginner**

Objective: Confirm the ACL authorizer is correctly configured and running. List all current ACLs, and verify that an unauthenticated client is denied access to a topic.

Step 1: Verify the Authorizer is Active

Check the broker startup log for the authorizer initialization message:

```
grep -i 'authorizer\|AclAuthorizer\|StandardAuthorizer' /opt/kafka/logs/server.log |
head -10
```

Expected Output: You should see a line like: [INFO] Successfully configured kafka.security.authorizer.AclAuthorizer

What authorizer class name did you find in the logs:

_____

Step 2: List All Existing ACLs

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --list
```

Expected Output: On a fresh setup, this should return no output (empty ACL store) or only the super user implicit entry.

Step 3: Attempt Access Without ACLs

Create a client properties file for alice (no ACLs granted yet):

```
# alice-client.properties
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="alice" password="alice-secret";
```

Attempt to produce a message as alice:

```
echo "test message" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config alice-client.properties \
  --topic orders
```

Expected Output: Access should be DENIED. You will see: ERROR org.apache.kafka.common.errors.TopicAuthorizationException: Not authorized to access topics: [orders]

Step 4: Check the Authorizer Log

```
grep 'DENIED\|Denied\|is not Allowed' /opt/kafka/logs/kafka-authorizer.log | tail -5
```

Expected Output: You should see a line showing: Principal = User:alice is Denied Operation = WRITE on Topic orders

Record what you observed in the authorizer log:

Log entry: _____
_____
_____

Lab 1 Completion Checklist

- ☐ Confirmed authorizer class is loaded in broker logs
- ☐ Listed all ACLs (empty on fresh cluster)
- ☐ Produced a TopicAuthorizationException for alice on topic 'orders'
- ☐ Found the DENIED entry in kafka-authorizer.log

LAB 2: Producer ACL — Grant and Verify Write Access

| Duration: 25 minutes | Level: **Beginner**

Objective: Grant User:alice Write and Describe permissions on the 'orders' topic. Verify she can produce messages, then remove the ACL and verify access is revoked.

Step 1: Grant Producer ACLs to alice

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --add \
  --allow-principal User:alice \
  --operation Write \
  --operation Describe \
  --topic orders
```

Expected Output: Adding ACLs for resource... Current ACLs for resource... 'Allow:User:alice' operations: [Write, Describe]

Step 2: Verify the ACL Was Created

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --list --topic orders
```

Record the output:

ACL list output: _____
_____
_____

Step 3: Test Producer Access as alice

```
kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config alice-client.properties \
  --topic orders
# Type a few messages: order-1, order-2, order-3 then Ctrl+C
```

Expected Output: Messages should be accepted without any authorization error. You will see the '>' prompt and messages will be sent successfully.

Step 4: Verify alice CANNOT Read (no Read ACL yet)

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --consumer.config alice-client.properties \
  --topic orders --from-beginning --max-messages 5
```

Expected Output: alice should receive TopicAuthorizationException or GroupAuthorizationException — she has Write but not Read, and no consumer group ACL.

Step 5: Remove the Write ACL and Confirm Revocation

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --remove \
  --allow-principal User:alice \
  --operation Write \
  --topic orders
# Confirm the removal when prompted: y
```

Now try producing again as alice:

```
echo "should fail" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config alice-client.properties \
  --topic orders
```

Expected Output: Access should be denied again — TopicAuthorizationException returns immediately.

Reflection Questions

7. Why is the Describe permission required alongside Write for a producer?

Answer: _____

8. What would happen if you grant Write but NOT Describe to a producer?

Answer: _____

Lab 2 Completion Checklist

- ☐ Granted Write + Describe ACLs to User:alice on topic 'orders'
- ☐ Verified ACL was created using --list
- ☐ Successfully produced messages as alice
- ☐ Confirmed alice cannot consume (no Read ACL)
- ☐ Removed the Write ACL and confirmed access revoked

LAB 3: Consumer ACL — Grant Topic + Group Access

| Duration: 30 minutes | Level: **Intermediate**

Objective: Configure a complete consumer ACL set for User:bob on topic 'orders' with consumer group 'order-processors'. Understand why BOTH topic and group ACLs are required.

Create bob's client properties file:

```
# bob-client.properties
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="bob" password="bob-secret";
```

Step 1: Ensure the Topic Has Messages

First, produce some messages as admin so bob has data to consume:

```
for i in $(seq 1 20); do echo "order-$i"; done | \
  kafka-console-producer.sh --bootstrap-server localhost:9092 \
  --producer.config admin-client.properties --topic orders
```

Step 2: Attempt Consumer Access Without ACLs (Observe Failure)

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --consumer.config bob-client.properties \
  --topic orders --group order-processors \
  --from-beginning --max-messages 5
```

Record the exact error message you receive:

Error message: _____
_____
_____

Step 3: Grant ONLY Topic Read ACL (Partial Fix)

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --add \
  --allow-principal User:bob \
  --operation Read \
  --operation Describe \
  --topic orders
```

Now try consuming again:

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --consumer.config bob-client.properties \
  --topic orders --group order-processors \
```

```
  --from-beginning --max-messages 5
```

Expected Output: Still fails — now with GROUP_AUTHORIZATION_FAILED. Topic ACL alone is not enough. Consumer must also have Group ACL.

Record the new error message:

New error message: _____
_____
_____

Step 4: Add the Consumer Group ACL

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --add \
  --allow-principal User:bob \
  --operation Read \
  --group order-processors
```

Step 5: Verify Complete Consumer Access

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --consumer.config bob-client.properties \
  --topic orders --group order-processors \
  --from-beginning --max-messages 10
```

Expected Output: bob should now consume messages successfully. The console will print the order messages you produced in Step 1.

Step 6: Verify bob Can Check His Own Consumer Group

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --command-config bob-client.properties \
  --describe --group order-processors
```

Expected Output: bob can describe his own group since he has Group:Read. He should NOT be able to list all groups (no Cluster:Describe).

Step 7: List and Audit All ACLs

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --list
```

Fill in the complete ACL table from the output:

Principal	Resource	Operation	Permission	Pattern
User:bob	Topic:orders	Read	Allow	LITERAL
User:bob	Topic:orders	Describe	Allow	LITERAL
User:bob	Group:order-processors	Read	Allow	LITERAL

Reflection Questions

9. Why does Kafka require a separate ACL for the consumer group in addition to the topic?

Answer: _____

10. What would happen if you granted Group:Read on '*' (wildcard) instead of a specific group?

Answer: _____

11. Can bob produce to the 'orders' topic with his current ACLs? Why or why not?

Answer: _____

Lab 3 Completion Checklist

- ☐ Observed consumer failure without any ACLs
- ☐ Added Topic Read+Describe ACL and observed GROUP_AUTHORIZATION_FAILED
- ☐ Added Group Read ACL and confirmed successful consumption
- ☐ Verified bob can describe his consumer group
- ☐ Filled in the complete ACL audit table

LAB 4: Prefixed ACLs for Multi-Tenant Topics

| Duration: 30 minutes | Level: **Intermediate**

Objective: Use PREFIXED ACL resource patterns to grant a service account access to all topics belonging to a team. Understand the efficiency benefits over individual LITERAL ACLs.

Step 1: Create Team-Namespaced Topics

```
# Create topics for team-payments
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --create \
  --topic team-payments-orders --partitions 2 --replication-factor 1
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --create \
  --topic team-payments-refunds --partitions 2 --replication-factor 1
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --create \
  --topic team-payments-notifications --partitions 2 --replication-factor 1
```

```
# Create a topic for team-logistics (different team)
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --create \
  --topic team-logistics-shipments --partitions 2 --replication-factor 1
```

Step 2: Create Prefixed ACL for payments-service

Instead of creating 3 separate ACLs for each payments topic, use a single PREFIXED ACL:

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --add \
  --allow-principal User:data-pipeline \
  --operation Write --operation Read --operation Describe --operation Create \
  --topic team-payments- \
  --resource-pattern-type PREFIXED
```

Expected Output: A single ACL rule now covers ALL current and FUTURE topics starting with 'team-payments-'

Step 3: Create Client Config for data-pipeline

```
# data-pipeline-client.properties
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
  username="data-pipeline" password="pipeline-secret";
```

Step 4: Test Access to All Payments Topics

```
# Test producing to team-payments-orders
echo "order-1" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config data-pipeline-client.properties \
  --topic team-payments-orders
```

```
# Test producing to team-payments-refunds
echo "refund-1" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config data-pipeline-client.properties \
  --topic team-payments-refunds
```

```
# Test producing to team-payments-notifications
echo "notification-1" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config data-pipeline-client.properties \
  --topic team-payments-notifications
```

Expected Output: All three produce operations should succeed with a single PREFIXED ACL.

Step 5: Verify Isolation — data-pipeline Cannot Access logistics Topics

```
echo "shipment-1" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config data-pipeline-client.properties \
  --topic team-logistics-shipments
```

Expected Output: This should FAIL with TopicAuthorizationException. The PREFIXED ACL only covers 'team-payments-*', not 'team-logistics-*'.

Step 6: Add a New Topic and Verify Automatic Coverage

```
# Create a new payments topic (no new ACL needed)
kafka-topics.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --create \
  --topic team-payments-chargebacks --partitions 2 --replication-factor 1
```

```
# data-pipeline should immediately have access without any ACL change
echo "chargeback-1" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config data-pipeline-client.properties \
  --topic team-payments-chargebacks
```

Expected Output: Succeeds immediately — PREFIXED ACLs cover new resources automatically. This is the key efficiency advantage.

Comparison: LITERAL vs PREFIXED ACL Count

Approach	ACLs Required (4 topics, 3 operations)	ACLs if 10 new topics added
LITERAL ACLs (one per topic)	4 topics x 3 operations = 12 ACLs	12 + 30 = 42 ACLs
PREFIXED ACL (single rule)	1 ACL	Still 1 ACL

Reflection Questions

12. When should you use PREFIXED ACLs vs. LITERAL ACLs? What is the trade-off?

Answer: _____

13. How would you apply a Prefixed ACL to cover all Kafka Streams internal topics for an app with application.id='payment-processor'?

Write the kafka-acls.sh command here:
_____
_____
_____

Lab 4 Completion Checklist

- ☐ Created 4 team-prefixed topics (3 payments + 1 logistics)
- ☐ Created single PREFIXED ACL for data-pipeline on 'team-payments-'
- ☐ Verified access to all 3 payments topics
- ☐ Confirmed data-pipeline CANNOT access logistics topic
- ☐ Created a new payments topic and confirmed automatic ACL coverage

LAB 5: Troubleshoot Authorization Failures

| Duration: 35 minutes | Level: **Intermediate**

Objective: Given a set of broken authorization scenarios, use the authorizer log and kafka-acls.sh to diagnose the exact missing ACL and fix each issue. Practice the L3 support diagnostic workflow.

In this lab, you are the on-call Kafka engineer. Each scenario presents an error report. Your job is to: (1) reproduce the error, (2) identify the exact missing ACL from the authorizer log, (3) apply the fix, and (4) verify the resolution.

Scenario 1: Application Cannot Start — Metadata Fetch Failure

The payments application team reports their producer is failing to start. The error in their application log is:

```
WARN  org.apache.kafka.clients.NetworkClient: Error connecting to node orders [id: 0]
ERROR Error when sending a message: Not authorized to access topics
```

The payments-service principal is User:order-service. Their client config connects to the 'orders' topic.

14. Reproduce: Try to produce as order-service with NO ACLs:

```
echo "test" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config order-service-client.properties \
  --topic orders
```

15. Check authorizer log to find what is being denied:

```
grep 'order-service' /opt/kafka/logs/kafka-authorizer.log | tail -10
```

What does the log show is being denied?

Denied operation and resource: _____

16. Write the kafka-acls.sh command to fix this:

Fix command: _____
_____
_____

17. Verify the fix works:

```
echo "fixed-message" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config order-service-client.properties \
  --topic orders
```

Did the fix work? What messages appeared in the authorizer log after the fix?

Result after fix:

_____

Scenario 2: Consumer Commits Failing — Silent Data Loss Risk

A consumer running as User:bob on group 'inventory-sync' is consuming from topic 'inventory' but offset commits are failing. The consumer keeps re-reading the same messages on restart.

18. Create the inventory topic if it doesn't exist (as admin), produce some messages
19. Grant bob Topic:Read + Describe on 'inventory' (topic ACL only), then try to consume:

```
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --consumer.config bob-client.properties \
  --topic inventory --group inventory-sync \
  --from-beginning --max-messages 5
```

What error do you see?

Error message:

_____

20. Check the authorizer log for the group-related denial:

```
grep 'inventory-sync\|bob' /opt/kafka/logs/kafka-authorizer.log | tail -10
```

What group operation is being denied?

Denied group operation:

_____

21. Apply the complete fix (include all necessary ACLs):

Complete kafka-acls.sh commands: _____
_____
_____

Scenario 3: Admin Tool Failure — Broker Refused Connection

A monitoring tool running as User:alice is trying to list all consumer groups across the cluster but receives CLUSTER_AUTHORIZATION_FAILED.

22. Reproduce the error:

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --command-config alice-client.properties \
  --list
```

What is the exact error?

Error: _____

23. Which cluster-level operation is missing? Write the fix:

Missing operation and fix command: _____
_____
_____

24. Apply the minimum required ACL and verify the tool works

Scenario 4: DENY ACL Blocking a Legitimate User

User:alice was accidentally given a DENY ACL on Topic:orders. Even though she has an ALLOW ACL for Write, she cannot produce. Diagnose and remove the blocking DENY ACL.

25. First, create the conflicting DENY ACL (simulating the accident):

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --add \
  --deny-principal User:alice \
  --operation Write \
  --topic orders
```

26. Also ensure alice has an ALLOW Write ACL (from Lab 2 or re-add):

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --add --allow-principal User:alice \
  --operation Write --operation Describe --topic orders
```

27. Try to produce as alice — confirm it fails despite having ALLOW:

```
echo "should-fail" | kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --producer.config alice-client.properties --topic orders
```

Why does the ALLOW ACL not help here?

Explanation: _____

28. Remove the DENY ACL:

```
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties \
  --remove \
  --deny-principal User:alice \
  --operation Write \
  --topic orders
```

29. Verify alice can now produce successfully

Lab 5 Completion Checklist

- ☐ Diagnosed and fixed Scenario 1: Producer metadata failure
- ☐ Diagnosed and fixed Scenario 2: Consumer group commit failure
- ☐ Diagnosed and fixed Scenario 3: Cluster authorization failure

- ☐ Demonstrated DENY overriding ALLOW in Scenario 4 and removed the DENY ACL

LAB 6: ACL Challenge: Secure a Full Application Stack

| Duration: 40 minutes | Level: **Advanced**

Objective: Design and implement a complete ACL policy for a multi-service application. This lab tests your ability to apply all concepts from the module to a realistic production scenario.

The Scenario

You are setting up a new e-commerce platform on Kafka with the following components. No ACLs exist yet (allow.everyone.if.no.acl.defined=false). You must grant exactly the right permissions — no more, no less.

Service	Principal	Topics Accessed	Role
Order Producer App	User:order-producer	orders (write)	Produces new orders
Order Consumer App	User:order-consumer	orders (read), group: order-grp	Processes orders
Inventory Service	User:inventory-svc	inventory (read+write)	Reads & updates inventory
Analytics Pipeline	User:analytics	orders, inventory (read-only)	Reads for reporting
Kafka Admin Tool	User:kafka-admin	All topics (full admin)	Cluster administration
Monitoring Service	User:monitor	All topics (describe-only)	Health checks and lag

Part A: Design Your ACL Plan

Before writing any commands, fill in this design table:

Principal	Resource	Operation(s)	Pattern	Justification
User:order-producer			LITERAL	
User:order-producer			LITERAL	
User:order-consumer			LITERAL	
User:order-consumer			LITERAL	
User:inventory-svc			LITERAL	
User:analytics			LITERAL	
User:kafka-admin			LITERAL	

Principal	Resource	Operation(s)	Pattern	Justification
User:monitor			LITERAL	

> **Tip:** Tip: Refer to the Complete ACL Sets tables in Module 5 of the Study Material for the minimum required operations for each role.

Part B: Create All Topics

```
kafka-topics.sh --bootstrap-server localhost:9092 --command-config admin-
client.properties \
  --create --topic orders --partitions 4 --replication-factor 1
kafka-topics.sh --bootstrap-server localhost:9092 --command-config admin-
client.properties \
  --create --topic inventory --partitions 4 --replication-factor 1
```

Part C: Implement the ACL Plan

Write and execute the kafka-acls.sh commands for each service. Use the space below to write your commands before executing them:

order-producer ACLs:

Commands: _____
_____
_____

order-consumer ACLs:

Commands: _____
_____
_____

inventory-svc ACLs:

Commands: _____
_____
_____

analytics ACLs:

Commands: _____
_____
_____

kafka-admin ACLs:

Commands: _____
_____
_____

monitor ACLs:

Commands: _____
_____
_____

Part D: Verify All Access Patterns

Create client property files for each service account and run these verification tests:

Test	Command	Expected Result	Pass/Fail
order-producer produces to 'orders'	kafka-console-producer.sh ...	Success	
order-producer reads from 'orders'	kafka-console-consumer.sh ...	DENIED	
order-consumer reads from 'orders'	kafka-console-consumer.sh ...	Success	
inventory-svc produces to 'inventory'	kafka-console-producer.sh ...	Success	
analytics reads from 'orders'	kafka-console-consumer.sh ...	Success	
analytics produces to 'orders'	kafka-console-producer.sh ...	DENIED	
monitor describes all topics	kafka-topics.sh --describe ...	Success	
monitor produces to any topic	kafka-console-producer.sh ...	DENIED	

Part E: Audit and Final Review

```
# Generate a complete ACL report
kafka-acls.sh --bootstrap-server localhost:9092 \
  --command-config admin-client.properties --list > acl-audit-report.txt
cat acl-audit-report.txt
```

How many total ACL rules did you create?

Total ACL count:

_____

Are there any ACLs that could be simplified with PREFIXED patterns?

Simplification opportunities:

_____

Challenge Extension (Bonus)

If you finish early: Implement the analytics service using a SINGLE PREFIXED ACL instead of separate topic ACLs, given that all topics in this system follow the pattern 'ecom-*':

Single PREFIXED ACL command for analytics:
_____
_____
_____

Lab 6 Completion Checklist

- ☐ Completed the ACL design plan table before writing commands
- ☐ Created all required ACLs for all 6 service accounts
- ☐ Ran all 8 verification tests and recorded Pass/Fail results
- ☐ Generated the final ACL audit report
- ☐ Identified simplification opportunities

Module Assessment Quiz

20 questions | Passing score: 75% (15/20) | Time: 20 minutes

Section A: Multiple Choice (1 point each)

1. Which authorizer class should be used for Kafka in KRaft mode?

- a) kafka.security.authorizer.AclAuthorizer
- b) org.apache.kafka.metadata.authorizer.StandardAuthorizer
- c) kafka.security.auth.SimpleAclAuthorizer
- d) org.apache.kafka.acl.KRaftAuthorizer

Answer: _____

2. A producer is receiving TOPIC_AUTHORIZATION_FAILED. Which TWO operations does it minimally need?

- a) Read and Describe
- b) Write and Create
- c) Write and Describe
- d) Write and Read

Answer: _____

3. What does allow.everyone.if.no.acl.defined=false mean?

- a) Anonymous users can access topics with no ACL
- b) All access is denied unless an explicit ACL grants it
- c) Only super users can access topics
- d) ACLs must be defined for every operation

Answer: _____

4. A consumer with Group:Read on group 'my-group' connects. Which additional ACL is required?

- a) Cluster:Describe
- b) Topic:Write on the target topic
- c) Topic:Read and Topic:Describe on the target topic
- d) Topic:Describe only

Answer: _____

5. User:alice has ALLOW Write on Topic:orders (LITERAL) and DENY Write on Topic:* (LITERAL wildcard). What happens when she tries to produce to 'orders'?

- a) Succeeds — LITERAL takes priority over wildcard
- b) Denied — DENY always overrides ALLOW regardless of specificity
- c) Succeeds — ALLOW and DENY cancel out
- d) Depends on the order the ACLs were created

Answer: _____

Section B: True or False (1 point each)

6. A Kafka super user defined in super.users is subject to ACL checks.

True / False: _____

7. PREFIXED ACLs automatically apply to new topics created with the matching prefix.

True / False: _____

8. User:Alice and User:alice are treated as the same principal by Kafka's authorizer.

True / False: _____

9. You need a Group ACL even if you are using assign() (manual partition assignment) instead of subscribe().

True / False + Explanation:

_____

10. ACLs are stored in ZooKeeper when using KRaft mode.

True / False: _____

Section C: Short Answer (2 points each)

11. Write the complete kafka-acls.sh command to grant User:stream-app Read, Write, Describe, Create, and Delete permissions on all topics starting with 'stream-app-' using a single ACL.

Command: _____
_____
_____

12. A Kafka Streams application with application.id='reporting-engine' is getting authorization errors on its internal changelog topics. What ACL(s) would you add, and why?

Answer: _____
_____
_____

13. Describe the complete diagnostic workflow when an application reports TOPIC_AUTHORIZATION_FAILED. List each step in order.

Workflow steps: _____
_____
_____

14. Your cluster has 50 microservices each writing to their own topic. A new compliance requirement says a central audit service must read ALL topics. What is the most efficient ACL approach? What are the risks?

Answer: _____
_____
_____

15. What is the difference between Cluster:Create and Topic:Create operations? When would you use each?

Answer: _____
_____
_____

Answer Key

Q#	Answer	Reference
1	b) StandardAuthorizer	Module 3.1
2	c) Write and Describe	Module 4.2 / Module 5.1
3	b) All access denied unless ACL grants it	Module 3.1
4	c) Topic:Read and Topic:Describe	Module 5.2
5	b) Denied — DENY overrides ALLOW	Module 2.4 / Module 6.4
6	False — super users bypass ALL ACL checks	Module 3.1
7	True	Module 6.2
8	False — case-sensitive	Module 2.1
9	False — assign() does not need Group:Read	Module 5.2
10	False — KRaft stores ACLs in the metadata log	Module 3.2
11-15	See instructor answer sheet	Modules 4–8

Prepared by: Rajesh Pasham
Apache Kafka Training Program | Security Module | ACLs & Authorization | 2026